

FARO — Technical documentation and Canvas implementation

Book 3 of 3 · For whoever installs, develops or deploys FARO

Version: September 2026 · SHIFT 2026 Challenge · Gamification

What this book covers

How FARO is built, module by module, and how it connects to a real Canvas: first to a free Instructure account (what can be done today, in an afternoon), then to an institution's instance through LTI 1.3 (what requires the institution to take part).

Chapters 1 to 5 describe the code. Chapters 6 and 7 are step-by-step procedures. Chapter 8 says how to check everything works, and chapter 9 what is real and what is simulated.

Contents

1. Architecture
2. Code structure and conventions
3. Domain logic (`src/lib/`)
4. The Canvas layer in the extension (`src/data/canvas/`)
5. The backend (`server/`)
6. Canvas implementation, step by step: free account
7. Canvas implementation, step by step: institution with LTI 1.3
8. Verification and tests
9. What is real and what is simulated

Requirements

Tool	Version	For
Node.js	22.18 or later (or 24)	The backend runs TypeScript without compiling (<code>node src/index.ts</code>)
npm	9+	Extension dependencies
Chrome / Edge	current	Loading the extension
A Canvas account	Free-for-Teacher or institutional	Chapters 6 and 7
A Gemini API key	Optional; with real students, from a project with active billing	The AI mentor (chapter 6.9)

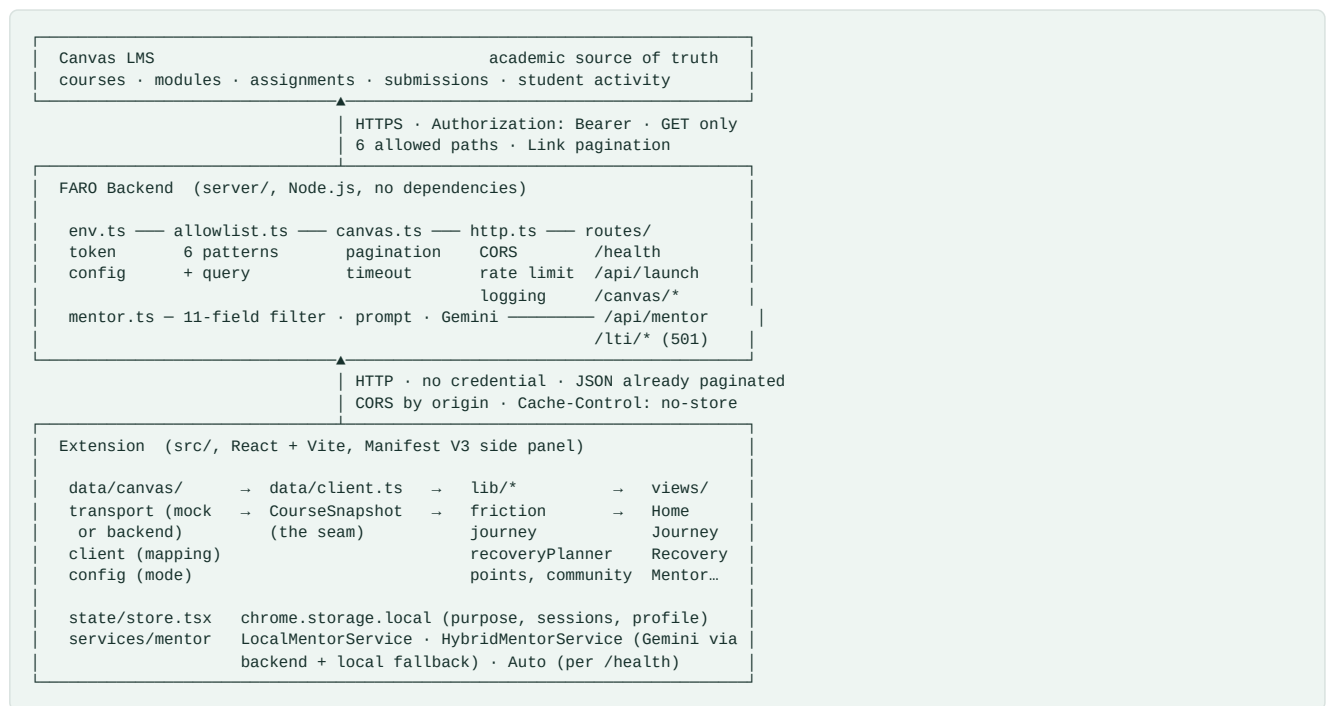
The commands, in one table

Where	Command	What it does
root	<code>npm install</code>	Installs the extension's dependencies (68 packages)
root	<code>npm run dev</code>	Vite dev server at <code>http://localhost:5173</code>
root	<code>npm run server</code>	The backend with auto-reload, from the root (= <code>npm --prefix server run dev</code>)

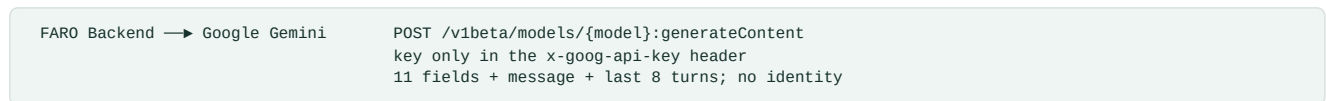
Where	Command	What it does
root	<code>npm run typecheck</code>	Checks types; if it prints nothing after the header, it passed
root	<code>npm run build</code>	Builds the extension into <code>dist/</code>
server/	<code>cp .env.example .env</code>	Creates the backend configuration (then edit it)
server/	<code>npm run dev</code>	Backend with auto-reload
server/	<code>npm start</code>	Backend
server/	<code>npm test</code>	38 checks against a fake Canvas and a fake Gemini
server/	<code>npm install</code>	Only if you want <code>npm run typecheck</code> there (installs <code>typescript</code> and <code>@types/node</code> ; the backend itself has no dependencies)

1. Architecture

1.1 The pieces



And, only when the AI mentor is on:



1.2 The four principles the shape follows

- 1. Canvas is the source of truth; FARO never copies or alters it.** Every FARO open re-reads the course. There is no grade database in FARO, and there will not be: what FARO persists is what FARO creates (purpose, sessions, preferences).
- 2. The credential lives in a single process.** The backend exists for this reason. An extension is public code; a token inside it is a published token. The backend holds the token, decides which paths it accepts and returns JSON. The extension does not know which token exists or what the header is called. The same rule holds for the Gemini key: it lives on the backend, and the extension only knows POST /api/mentor .
- 3. The seam is a type, not a class.** CourseSnapshot (src/data/client.ts) is the only way course data enters the rest of the application. Everything above it (lib/ , views/) works on that type and does not know whether it came from fixtures or an institution. That is why switching modes is an environment variable and not a rewrite.
- 4. The logic does not know React exists.** src/lib/ are pure functions: data in, a decision out. They can be tested without mounting the interface, and can move to the backend unchanged when the friction engine must run on the server.

1.3 The two modes

	mock	http
Chosen with	VITE_CANVAS_MODE=mock (default)	VITE_CANVAS_MODE=http
Who answers the Canvas paths	MockCanvasTransport with fixtures.ts	BackendCanvasTransport → backend → Canvas
Launch (course + person)	Fixed: course 4021, student 77120	GET /api/launch to the backend
Study profile (measured rhythm)	Fixture: 35 min/session, 4 days/week	Absent; Recovery says so

	mock	http
Minutes estimate per activity	Fixture table	20 min default for all
Needs a backend	No	Yes

What does **not** change between modes: `client.ts` (the mapping), `lib/` (all the rules), `views/` (all the screens), `i18n/`. It is the same code against a different transport.

The mentor has its own switch, independent of Canvas's:

	auto	local	gemini
Chosen with	<code>VITE_MENTOR_MODE=auto</code> (default)	<code>VITE_MENTOR_MODE=local</code>	<code>VITE_MENTOR_MODE=gemini</code>
Who answers open conversation	<code>AutoMentorService</code> : asks <code>GET /health</code> (<code>probeGemini</code> , 2 s, cached 30 s, no student data). If the server reports a mentor, as <code>gemini</code> ; if it does not answer or has no key, <code>LocalMentorService</code>	<code>LocalMentorService</code> , on the device	<code>HybridMentorService</code> : <code>HttpMentorService</code> → <code>POST /api/mentor</code> → Gemini; on failure, <code>LocalMentorService</code>
What leaves the device	The <code>/health</code> check; with Gemini, the same as <code>gemini</code>	Nothing (not even the <code>/health</code> check for the mentor)	The 11-field context, the message and the last 8 turns, to the backend
Needs a backend	No; uses it if it has <code>GEMINI_API_KEY</code>	No	Yes, with <code>GEMINI_API_KEY</code> (Canvas may be unconfigured)

With `auto`, putting the key in `server/.env` and starting the server is enough: the extension uses Gemini with no rebuild and no `.env.local`. `MentorView` repeats the check every time it opens, and the footer says who answers based on that result. An institution that does not approve the AI mentor builds with `VITE_MENTOR_MODE=local` (or gives the server no `GEMINI_API_KEY`).

In all three modes, the mentor's structured moments (check-in, Focus, Way back, weekend plan, points, local Teach me) are computed in the extension with `lib/`; a model never decides them.

1.4 The flow of one open

1. `main.tsx` mounts `App` inside `StoreProvider`.
2. `store.tsx` reads `chrome.storage.local` (purpose, sessions, preferences, profile, the saved `weekPlan`).
3. `store.tsx` calls `faroClient.getCourseSnapshot()`.
4. `CanvasFaroClient` resolves the launch (`resolveLaunch`), issues the four requests in parallel and maps the responses to `CourseSnapshot`.
5. `store.tsx` derives everything else with `lib/`: friction, journey, next best action, recovery plan, points, achievements and the rhythm week (`week`).
6. `App.tsx` shows Purpose if there is no destination; otherwise the active tab.
7. If step 3 fails, `App.tsx` shows the error with a `Retry` button. A course is never invented.

1.5 What the backend does and does not do

Does: hold the token and the Gemini key, validate paths, talk to Canvas, paginate, rate-limit, restrict origins, log without personal data, resolve the launch in the pilot, and answer `POST /api/mentor` by calling Gemini with a re-checked context (chapter 5). Canvas and the mentor are separately optional: the backend can run mentor-only.

Does not do yet [pending]: persist FARO state (PostgreSQL), run the friction engine, store conversations (on purpose: they are not stored), verify LTI launches (the routes answer 501 with the specification in comments).

The rule for moving something to the backend: when it needs data from more than one device, when it needs a credential, or when it must be auditable by the institution. Until then, it stays in the extension.

2. Code structure and conventions

2.1 The tree

faro/	
├─ public/	
│ └─ manifest.json	Manifest V3: side panel, storage; host_permissions localhost only
│ └─ background.js	minimal service worker: opens the side panel
├─ src/	
│ └─ App.tsx	routing between views, bottom bar, loading and error states
│ └─ main.tsx	entry point
│ └─ types/index.ts	ALL domain types, in one file
│ └─ data/	
│ └─ client.ts	CourseSnapshot + faroClient: the seam
│ └─ canvas/	the Canvas layer (chapter 4)
│ └─ endpoints.ts	the 6 paths, the scopes, the audit surface
│ └─ raw.ts	types as Canvas returns them (snake_case)
│ └─ transport.ts	MockCanvasTransport · BackendCanvasTransport · canvasLog
│ └─ client.ts	CanvasFaroClient: the real mapping
│ └─ config.ts	mode, backend URL, launch resolution
│ └─ status.ts	behaviour flags (activity fallback)
│ └─ fixtures.ts	the demo course, in Canvas's exact shape
│ └─ community.mock.ts	
│ └─ reviewBank.ts	question bank per module (review cards and local Teach me)
│ └─ rewards.mock.ts	the semester's simulated previous courses
│ └─ lib/	pure logic, no React and no fetch (chapter 3)
│ └─ timeSession.ts	buildTimeSession · planDays · openSteps · sessionRules
│ └─ rewardPath.ts	pathToPoints: the realistic path to the next milestone
│ └─ teach.ts	teachTopics · lessonQuestions · answerTeach
│ └─ rhythm.ts	learningRhythm · weekRhythm · activeDays
│ └─ ...	friction, journey, recoveryPlanner, points, rewards, mentorContext...
│ └─ services/mentor.ts	Local · Http · Hybrid · AutoMentorService, probeGemini
│ └─ state/	
│ └─ store.tsx	the app state and its derivations
│ └─ storage.ts	chrome.storage.local with a localStorage fallback
│ └─ community.tsx	Community state
│ └─ theme.tsx	light/dark
│ └─ views/	one view per screen
│ └─ components/	reusable pieces (CanvasPanel, Cards, FeedPost...)
│ └─ i18n/	es.ts (source of truth) · en.ts (typed against es) · index.ts
│ └─ styles/	tokens.css · global.css · views.css (plain CSS)
└─ server/	the backend (chapter 5)
└─ src/	
└─ index.ts	start-up
└─ app.ts	assembles routes
└─ env.ts	configuration and .env
└─ allowlist.ts	the 6 allowed paths
└─ canvas.ts	the only function that talks to Canvas
└─ http.ts	router, CORS, rate limit, logging, JSON body ≤ 32 KB
└─ mentor.ts	sanitizeMentorRequest · systemPrompt · callGemini
└─ routes/	health · launch · canvas · mentor · lti
└─ test/	
└─ fake-canvas.ts	a fake Canvas with real pagination
└─ fake-gemini.ts	a fake Gemini (replies, errors, slowness)
└─ e2e.ts	38 checks
└─ .env.example	
└─ package.json	"dependencies": {}
└─ docs/	the three books
└─ .env.example	VITE_CANVAS_MODE, VITE_FARO_API_URL, VITE_MENTOR_MODE
└─ package.json	react, react-dom; vite, typescript; `server` script

2.2 The three seams that matter

lib/ does not import React. If a file in `lib/` needs `useState`, the logic is in the wrong place. The proof: `src/lib/` compiles with `tsc` without `@types/react`.

types/index.ts separates Canvas from FARO. The `Canvas*` types mirror the API. The rest are ours. A field `Canvas` does not have (`estimated_minutes`) carries a comment saying so.

i18n/es.ts is the source of truth. `en.ts` is declared as `const en: Dict` against the Spanish type. A key added in one language and not the other **breaks the build**. That is intentional.

2.3 Conventions

Convention	Rule	Why
Thresholds	Named constants in an exported object (<code>frictionRules</code> , <code>pointsConfig</code> , <code>feasibilityRules</code> , <code>sessionRules</code>)	Tuned in one place and can be shown in documentation
Copy	Always from <code>t.*</code> (the active dictionary), never literals in views	Two languages, and the compiler watches
Components	Small; a <code>.tsx</code> over ~300 lines probably contains <code>lib/</code> logic	Readability
URLs	No hand-written API URL outside <code>data/</code> and <code>server/</code>	One place to change
Comments	Explain <i>why</i> , not <i>what</i> . Every non-obvious design decision carries its reason next to the code	Whoever comes next understands without asking
Styles	Plain CSS with tokens in <code>tokens.css</code> ; no frameworks	Small bundle; the extension must feel immediate
TS syntax in <code>server/</code>	Erasable syntax only (<code>erasableSyntaxOnly</code>): no <code>enum</code> , no constructor parameter properties	Node runs the TypeScript directly without compiling
Imports in <code>server/</code>	With an explicit <code>.ts</code> extension	Node requirement for running TS

2.4 Alias

`@/` points to `src/` (configured in `vite.config.ts` and `tsconfig.json`). It is an alias relative to Vite's root, so it works identically on Windows and macOS without path helpers.

2.5 What is NOT there, on purpose

- No Next.js, Electron, Redux or Tailwind. Project instructions: the extension must be lightweight.
- No ORM or database yet.
- No HTTP framework in the backend (Fastify was planned; it was replaced by `node:http` so the token-holding process runs no third-party code; the routes have the shape `(req) => reply` , which ports to Fastify unchanged if the project decides to).
- No test dependency. The backend tests use `node:assert` and run with `node` .
- No Google SDK. The backend calls Gemini with native `fetch` .

3. Domain logic (src/lib/)

Everything FARO *decides* lives here, in pure functions. This chapter documents each module with its inputs, its exact rules and its outputs. The numbers are those in the code; if the code changes, this chapter must change with it.

3.1 friction.ts — the friction engine

Input: a `CourseSnapshot` and whether the student is returning after time away. **Output:** a `FrictionSignal` with state, days without activity, pending count, momentum and a supportive headline.

Thresholds (`frictionRules` , configurable):

Constant	Value	Use
<code>frictionDays</code>	3	From here on, FRICTION
<code>disconnectionDays</code>	7	From here on, DISCONNECTION
<code>overwhelmPendingCount</code>	4	Overdue pending items for POSSIBLE_OVERWHELM
<code>momentumDecayWindow</code>	14	Days of inactivity that take momentum from 100 to its floor
<code>momentumFloor</code>	20	Momentum never goes below this

States, in evaluation order:

returning	→ RECOVERY
days ≥ 7	→ DISCONNECTION
pending ≥ 4 and days ≥ 3	→ POSSIBLE_OVERWHELM
days ≥ 3	→ FRICTION
otherwise	→ FLOWING

Days without activity: the most recent timestamp in `snapshot.activity` . If there is none, `disconnectionDays` (7) is assumed: with no evidence of activity, FARO treats the student as disconnected, not as on track.

Pending: assignments with no submission whose due date has passed.

Momentum:

```
decay = min(1, days / 14)
load  = min(1, pending / 8) × 0.2
raw   = (1 - decay) × 100 - load × 100
momentum = max(20, round(raw))
```

One day without activity costs ~7 points. Eight overdue items subtract 20. Nothing takes it to zero.

Product rule: the `headline` comes from the dictionary per state and is always supportive language. There is no "risk" field.

3.2 journey.ts — the route and the next best action

`buildJourney(snapshot, friction)` orders assignments by module position and then by id, and assigns each a state:

has a submission	→ completed
no submission and due date passed	→ missed
no submission and module locked	→ locked
the first one that is none of the above	→ current
the rest	→ upcoming

The last assignment of each module is a **checkpoint** (`checkpoint: true`). `progressPercent` = completed ÷ total. `currentIndex` is the first `missed` or `current` . `recalculated` is true when the friction state is not FLOWING.

`nextBestAction(journey, availableMinutes)` — the four-step rule:

1. Open = `missed` + `current` + `upcoming` .
2. Of those, the ones that fit in `availableMinutes` .
3. Of those, the shortest.
4. If none fits, the shortest of all open ones.

The reason shown depends on the case: *reopens the route* if it is `missed`, *fits your time* if it fits, *the shortest* otherwise.

`comebackMilestone(journey)` — the shortest open one, with no time condition. It is the comeback mission: not the most important, but the one least likely to be postponed again.

`recoveryRoute(journey)` — three steps (today / tomorrow / next session) taken from the route.

3.3 `recoveryPlanner.ts` — feasibility and plan

This module answers "*can I still finish?*" with numbers.

`remainingWork(snapshot, journey)` — everything not completed, **including locked modules**. Ignoring them would be the reassuring lie this file exists to avoid. Returns activities, minutes, modules, days until the close and the close date.

`capacityFor(stated, profile)` — what the student sustains per day. What they declared wins; if nothing was declared, the measured rhythm spread across the week: $(\text{minutesPerSession} \times \text{daysPerWeek}) \div 7$, rounded to 5. With neither, 0.

`feasibility(work, capacity)` — the verdict:

```
activities = 0                                → comfortable (finished)
no deadline, or daysLeft ≤ 0, or capacity ≤ 0 → unknown (no guessing)
required = minutes / daysLeft
load      = required / capacity
load ≤ 0.85                                → comfortable
load ≤ 1.40                                → tight
otherwise                                     → not_realistic
```

`feasibilityRules = { comfortableLoad: 0.85, stretchLoad: 1.4 }`. `not_realistic` exists on purpose.

`strategies(work, capacity)` — three paces over the same work, with `PACE = { comfortable: 1, balanced: 1.35, intensive: 2 }` multiplying the required daily minutes. Each strategy computes its days needed and its slack (`buffer = daysLeft - days`). **At most one** carries `suggested: true`: the feasible one closest to the student's capacity. With no known capacity, none is suggested.

`buildPlan(journey, work, dailyMinutes, strategy)` — packs the work into days:

- Priority: `missed` first; then nearest due date; then course order.
- Minimum budget per day: 10 minutes.
- Front-loaded: finishing early leaves the slack *before* the close.
- An activity longer than a whole day is scheduled anyway, taking its own day.
- What does not fit before the close goes to `overflow`. It is shown, never silently dropped.

`todayOneThing(plan)` — the first item of the first day.

3.4 `lifeHappened.ts` — the life state and the intervention

Six states, each with an intervention kind and a minute budget:

State	Intervention	Minutes
<code>less_time</code>	<code>shrink_session</code>	10
<code>overwhelmed</code>	<code>single_action</code>	15
<code>dont_understand</code>	<code>mentor</code>	15
<code>lost_routine</code>	<code>comeback_mission</code>	10
<code>need_break</code>	<code>pause</code>	keeps the current one
<code>ready</code>	<code>continue</code>	20

Nothing here touches dates or academic rules. It only changes the size and shape of the next step.

3.5 `points.ts` and `rhythm.ts` — points

`pointsConfig` (semester scale):

Constant	Value
POINTS_PER_ACTIVITY	50 per completed activity on the route (submitted in Canvas or finished in FARO)
POINTS_PER_MODULE	250 per module with all its activities completed
STREAK_BONUS	25 (every 3 consecutive days, <code>rhythmMilestoneEvery = 3</code>)
COMEBACK_BONUS	125 (once, if there is at least one FARO session and <code>awayGap ≥ frictionDays</code>)
REVIEW_SET_BONUS	75 per finished set of review cards

`computePoints` also returns `thisWeek`: what was earned in the last 7 days through actions taken in FARO (sessions, cards, and the comeback bonus if one of them earned it).

`learningRhythm(sessions)` counts consecutive days with a session, ending today, and only feeds the rhythm bonus: a day without a session makes it stop adding; it never subtracts. What the student sees is not that count but the `weekRhythm` week (3.13).

3.5b `rewards.ts` — the semester reward

`rewardsConfig = { SEMESTER_GOAL: 10_000, SEMESTER_CAP_MXN: 200 }` and three tiers on a single track:

Tier	Points
\$50 MXN	2,500
\$100 MXN	5,000
\$200 MXN	10,000

- `semesterStatus({ currentCourse, previousCourses, redemption })` → total, percentage (capped at 100), tier reached, next tier and points to go.
- Tiers **unlock, they are not bought**: reaching one deducts nothing.
- `canRedeem(tier, status)` — the tier is reached and nothing was redeemed this semester. **One redemption per semester**: redeeming closes the reward at that tier.
- `remainingCoursePoints(journey)` — the minimum finishing the current course is worth (remaining activities + modules; bonuses come on top).

The semester's previous courses live in `data/rewards.mock.ts` (simulated, labelled on screen). Demo calibration: 7,700 from previous courses + 850 from the current course = 8,550 (85%); finishing the remaining activities and modules (1,450) reaches exactly 10,000.

3.5c `reviewCards.ts` — reconnection cards

- `touchedModules(journey)` — modules with at least one completed activity, most recent first.
- `pickReviewSet(journey, bank)` — 3 questions: up to 2 from the most recent module, then the one before. Deterministic. Returns `[]` when there are not enough (the offer is hidden; questions are never invented).
- `shouldOfferReview({ awayGap, frictionDays, questions, records })` — offered if `awayGap ≥ 3`, there are 3 questions and no set was completed today.
- `reviewedToday(records)` — one rewarded set per day.

The bank (`data/reviewBank.ts`) is bilingual and tied to the demo course (`REVIEW_BANK_COURSE_ID`): a real course with no bank gets no cards. In production, the bank is written by the instructor or generated by the backend from the module's pages; **a student token cannot read Canvas quiz questions**, and a reconnection card must never be confused with a graded quiz.

3.6 `achievements.ts` — the five achievements

Id	Exact condition
<code>the_comeback</code>	<code>awayGap ≥ 3</code> and at least one session exists
<code>back_on_track</code>	at least 2 sessions and <code>momentum ≥ 65</code>
<code>weathered_the_storm</code>	some session with <code>pendingAtStart ≥ 2</code>
<code>smart_session</code>	some session with <code>fitAvailableTime</code>

Id	Exact condition
finisher	progressPercent ≥ 100

While unearned, `hint` is shown, never a reproach.

3.7 `sessions.ts` — FARO's own record

A `StudySession` is what FARO notes down when the student marks a step as done. It stores minutes, how many pending items there were at the start and whether it fitted the declared time. `applySessions` overlays the sessions on the route so momentum, points and achievements respond instantly, without waiting for Canvas to record the submission.

3.8 `community.ts` — feed ranking and SOS routing

`rankFeed(posts, ctx)` orders by: relevance to the student's course and module, whether someone is asking for help and unanswered, and recency. **It ignores reactions on purpose.** `KIND_WEIGHT` puts `help: 5` above `achievement: 2`.

`sosRouting`: `topic → mentor`, `time → recovery`, `people → community`. Three kinds of stuck, three systems.

`communityPointsConfig`: `POINTS_PER_HELPFUL_MARK: 25`, `POINTS_PER_COMPLETED_ROOM: 100`. Joining a room gives no points; finishing it does.

3.9 `mentorContext.ts` — the privacy boundary

One function, `buildMentorContext`, that produces the eleven fields of `MentorContext`. If a piece of data is not built here, the mentor does not receive it. It does not read `Profile`. It is the only file Book 1 cites as the mentor's data contract. When Gemini answers, the backend applies the same contract again (`sanitizeMentorRequest`, chapter 5): the extension is not the only barrier.

3.10 `timeSession.ts` — "how much time do you have?"

Turns a number of minutes into a session. Used by Home's picker (5 / 10 / 20 / 30+), the Mentor's Focus mode (5 / 10 / 15 / 30) and the weekend plan, so the three always agree.

`sessionRules = { REVIEW_MINUTES: 3, OPEN_ENDED_MINUTES: 45 }` — the review counts as 3 minutes; the "30+" option plans 45.

`openSteps(journey)` — open activities in the order FARO would do them: `missed` first, then `current` and `upcoming` in route order. Locked ones are left out.

`buildTimeSession(journey, minutes, { reviewAvailable })`:

```

reviewAvailable and minutes ≥ 3      → first { review, 3 min }
for each step of openSteps, in order:
  fits in what is left                → added
  does not fit                        → stop (never skips to a shorter later one)
  no step added and ≥ 5 min left      → { partial: "the first N min" of the first step }
```

It never skips ahead: a module's quiz does not come before the exercise it checks. `sessionMinutes(items)` adds the session up. Minutes are FARO's estimates; the UI says "about".

`planDays(journey, days)` — e.g. Saturday 60 and Sunday 45. Steps go in route order and are never repeated across days; a day stops at the first step that does not fit, so the plan stays smaller than the time, on purpose. A step longer than an empty day's whole budget still takes that day rather than disappearing. A day with 0 minutes stays empty. The result is a `DayPlan[]`; if the student accepts it, it is saved as a `WeekPlan` with `setWeekPlan` (storage key `weekPlan`) and shows on Home.

3.11 `rewardPath.ts` — the realistic path to the next milestone

`pathToPoints(journey, gap)` answers "what do I need for the next TecmiRewards milestone?" with the same numbers as Impact, never with a model's.

- Walks everything not completed in route order, `missed` first. **Locked steps are included:** they open as the route advances, and leaving them out would understate the course.
- Each step adds `POINTS_PER_ACTIVITY` (50); if it is its module's last open step, it also adds `POINTS_PER_MODULE` (250).

- Stops when the points cover `gap`.
- Returns `{ steps, modules, points, minutes, reachable }`. `reachable: false` when finishing everything left in the course still falls short; the Mentor says so, without dressing it up.

Only guaranteed points count. Rhythm, comeback and review-card bonuses would shorten the path, but promising them would be a nudge dressed up as a fact.

3.12 `teach.ts` — Teach me without a model

The loop: **explain briefly** → **one question** → **answer** → **adapt**. The review bank (`data/reviewBank.ts`) already holds, per module, a question, its options and a short explanation: a micro-lesson.

- `teachTopics(journey, bank, moduleNames)` — modules with questions; the ones the student has touched (`touchedModules`) first, then the rest in course order.
- `lessonQuestions(bank, moduleId)` — the module's questions, in order.
- `answerTeach(bank, state, option)` with `TeachState = { moduleId, index, tried }`:

```
right option → { right, next: the module's next question | null }
wrong option → { wrong, question, remaining: options without those already tried }
```

With `next`, the Mentor asks the second, slightly different question; with `null`, it suggests applying it to the next step. With `wrong`, it repeats the key idea (`explain`) and asks again without the option already tried. No points are at stake.

Lessons exist only for the bank's course (`REVIEW_BANK_COURSE_ID`). When Gemini is available, open topics go to the backend with `mode: 'teach'` (the prompt's TEACH MODE, chapter 5).

3.13 `rhythm.ts` — the rhythm week

`weekRhythm(sessions, activity, now)` — the current week, Monday to Sunday, as seven `RhythmDay { weekday (0 = Monday), date, active, today, future }`. A day is `active` if FARO recorded a session **or** Canvas recorded course activity that day. Days after today are `future`, not "missed".

`activeDays(week)` — how many active days the week has.

The store exposes `week` (derived from the sessions and `snapshot.activity`). It feeds Home's rhythm tile (" $N/7 \cdot \text{active days this week}$ "), Progress's *Your rhythm* card and Impact's "*Studied N days this week*" line.

Product rule: flexible consistency. For someone who works, five days out of seven is a great week, and a gap on Wednesday breaks nothing. There is no streak to lose.

4. The Canvas layer in the extension (`src/data/canvas/`)

Seven files. It is the only part of the extension that knows Canvas exists.

4.1 `endpoints.ts` — the audit surface

The paths FARO uses, each built from one function:

```
course(id)           GET /api/v1/courses/:id
modules(id)          GET /api/v1/courses/:id/modules?include[]=items&per_page=100
assignments(id)       GET /api/v1/courses/:id/assignments?include[]=submission&per_page=100
userActivity(id, uid) GET /api/v1/courses/:id/analytics/users/:uid/activity
activeCourses()       GET /api/v1/courses?enrollment_state=active&per_page=100
self()               GET /api/v1/users/self (backend only, when resolving the launch)
```

And two lists for the Canvas administrator: `requiredScopes` in the developer key's `url:GET|/api/v1/...` form, and `ltiScopes` with what FARO requests (`contextmembership.readonly` , `result.readonly`) and what it refuses (`score` , `lineitem`).

`PER_PAGE = 100` is Canvas's maximum: fewer round trips, same data.

4.2 `raw.ts` — Canvas's shape

`snake_case` types that declare **only the fields FARO reads**. Canvas sends many more; they are ignored rather than modelled, so a new Canvas release that adds fields cannot break this layer. When this file and the Canvas documentation disagree, the difference should be visible: that is why it is not translated into the code style.

Important fields:

- `RawModule.items` — present when `include[]=items` was requested. Each item has `content_id` (the id of the assignment it points at) and `completion_requirement.completed` (whether *this* student met it).
- `RawModule.state` — `locked` | `unlocked` | `started` | `completed`. **Only present if the caller is a student in the course.** An instructor does not see it.
- `RawAssignment.submission` — present with `include[]=submission`. A `workflow_state: 'unsubmitted'` is a stub that means *not done*.
- `RawUserActivity.page_views` — an object whose keys are ISO dates rounded to the hour and whose values are counts. FARO keeps the keys and discards the values.

4.3 `transport.ts` — how the request is made

One interface, two implementations:

```
interface CanvasTransport {
  readonly mode: 'mock' | 'http'
  get<T>(path: string): Promise<T>
}
```

MockCanvasTransport — looks the path up in a list of patterns (`fixtureRoutes` in `client.ts`), waits 140 simulated ms and returns the fixture. A path with no pattern produces **404 in the log**, exactly like a wrong path against a real Canvas. A typo in `endpoints.ts` shows; it is not hidden.

BackendCanvasTransport — does `fetch( ${apiUrl}/canvas${path} )` with no authorisation header. The backend paginates; here a single round trip is the whole answer. A non-2xx status throws `CanvasHttpError` with the code and the `error` from the backend's JSON body (`path_not_allowed` , `canvas_error` , `canvas_unreachable` ...).

canvasLog — an in-memory ring buffer of 30 records (`at` , `method` , `path` , `mode` , `status` , `ms` , `items` , `error`). The Profile panel subscribes and draws it. It is never persisted or sent. It is the difference between claiming an integration and showing one.

4.4 `client.ts` — the mapping

`CanvasFaroClient.getCourseSnapshot()`:

1. Resolves the launch (`courseId` , `userId`).

2. Issues the four requests in parallel with `Promise.all`.
3. If the activity one fails with 401/403, not everything fails: it sets `integrationStatus.activityFallback = true` and returns `null` for that part. Any other error does propagate.
4. Walks the modules and their items to build two maps: `moduleOfAssignment` (from `content_id` to module id, only for `Assignment` or `Quiz` items) and `completedByModule`.
5. Filters the assignments down to those some module item points at. **An assignment with no module item is not on the route:** Canvas allows loose assignments, and putting them on the journey would invent structure the teacher did not create.
6. Maps each assignment: `submission` is kept only if `workflow_state !== 'unsubmitted'` and there is a `submitted_at`. That is: **done is what the submission says, not the due date.**
7. Maps activity to timestamped events; or, in the fallback, derives events from the student's own submission dates.
8. `estimated_minutes` comes from the injected estimate function: the fixture table in mock mode, `undefined` (→ 20 min default) in http mode, because fixture ids would match real ids by coincidence.
9. `profile` (measured rhythm) exists only in mock mode. Live, FARO is honest about not knowing the student's rhythm yet.

Three decisions a careless integrator would get backwards, named at the top of the file: module membership comes from the items, not the assignment; "done" comes from the submission, not the date; estimated minutes are FARO's and are stated as such.

4.5 `config.ts` — mode and launch

```
canvasConfig = {
  mode: VITE_CANVAS_MODE === 'http' ? 'http' : 'mock',
  apiUrl: VITE_FARO_API_URL ?? 'http://127.0.0.1:3000',
}
```

There is no token field and there cannot be one.

`resolveLaunch()` returns the `LaunchContext` (course, name, person, verified, source). In mock it is immediate (source: 'fixture'). In http it calls `GET /api/launch` once and caches it; `currentLaunch()` and `onLaunch()` let the Profile panel update when it arrives.

`verified` is `false` in both modes today. Only a signed LTI launch will set it to `true`, and the panel says so.

4.6 `status.ts` — behaviour flags

An observable object with `activityFallback`. It exists so a fallback does not pass silently as if it were the real thing: the Profile panel shows the note *"your account cannot read Canvas analytics, so activity is derived from the dates of your own submissions"*.

4.7 `fixtures.ts` — the demo course

A course in Spanish, *Gestión de Proyectos* (GP-101), 6 modules, 16 activities, in the exact shape Canvas sends. The scenario: an adult five weeks into a course, who stopped logging in `DAYS_INACTIVE = 5` days ago, with `DAYS_UNTIL_COURSE_ENDS = 12` days until the close. It is the exact moment FARO exists for.

With these fixtures the expected numbers are: 16 activities, 7 completed (44%), 9 remaining (4 h 35 min), 12 days, 5 without activity, module 3 started, module 5 locked, state FRICTION.

4.8 Switching modes

```
# .env.local in the root (git ignores it)
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

`npm run dev` or `npm run build`. No code file changes.

5. The backend (server/)

A Node.js process, with no dependencies, that holds the Canvas token and the Gemini key, and serves the FARO API. It runs TypeScript directly (`node src/index.ts` , Node 22.18+). Canvas and the mentor are separately optional, but at least one must be configured.

5.1 Configuration (env.ts)

Read once at start-up, from `process.env` with a fallback to `server/.env` . The process **refuses to start**:

- with half a Canvas configuration (`CANVAS_API_URL` without `CANVAS_ACCESS_TOKEN` , or the other way round);
- with nothing to serve (neither Canvas nor `GEMINI_API_KEY`);
- with `CANVAS_API_URL` or `GEMINI_API_BASE` not using `https://` (except localhost).

With only `GEMINI_API_KEY` , it starts **mentor-only**: `/api/launch` and `/canvas/*` answer 503 `canvas_not_configured` , and the extension stays in `mock` mode for Canvas.

Variable	Default	What it is
<code>CANVAS_API_URL</code>	empty (optional, paired with the token)	<code>https://canvas.instructure.com</code> or <code>https://<institution>.instructure.com</code>
<code>CANVAS_ACCESS_TOKEN</code>	empty (optional, paired with the URL)	Token of the student account
<code>FARO_COURSE_ID</code>	empty	Course to open; if absent, the first active enrolment
<code>FARO_HOST</code>	<code>127.0.0.1</code>	Listening interface
<code>FARO_PORT</code>	<code>3000</code>	Port
<code>FARO_ALLOWED_ORIGINS</code>	<code>http://localhost:5173,http://127.0.0.1:5173</code>	Allowed browser origins, comma-separated
<code>FARO_STRICT_ORIGINS</code>	empty (<code>false</code>)	<code>true</code> = accept only <code>FARO_ALLOWED_ORIGINS</code> , on loopback too (see 5.4)
<code>FARO_RATE_LIMIT_PER_MINUTE</code>	<code>120</code>	Requests per minute per client
<code>FARO_CANVAS_TIMEOUT_MS</code>	<code>10000</code>	Maximum time for one Canvas call
<code>FARO_LOG_LEVEL</code>	<code>info</code>	<code>info</code> or <code>debug</code>
<code>GEMINI_API_KEY</code>	empty (optional)	Gemini API key. Empty = mentor off; the extension uses its local mentor
<code>GEMINI_MODEL</code>	<code>gemini-3.5-flash</code>	Model code. Names change: check <code>ai.google.dev/gemini-api/docs/models</code>
<code>GEMINI_TIMEOUT_MS</code>	<code>20000</code>	Maximum time for one Gemini call
<code>FARO_MENTOR_PER_MINUTE</code>	<code>20</code>	Model calls per minute per client (cost control), on top of the global limit
<code>GEMINI_API_BASE</code>	<code>https://generativelanguage.googleapis.com</code>	Tests only: point at a fake Gemini

`describe(env)` is the only representation of the configuration that may be printed or returned: Canvas host (or `null`), port, origins, `tokenPresent` and `mentor: { provider: 'gemini', model } (or null)` . Never the token or the key.

5.2 Allow list (allowlist.ts)

Six patterns, each with its `purpose` written next to it. `allow(pathname, search)` :

1. Decodes the path (`%2e%2e` cannot escape).
2. Rejects `..` and `//` .
3. Looks for a matching pattern; if none, `path_not_allowed` .
4. Rebuilds the query with allowed keys only (`include[]` , `per_page` , `enrollment_state` , `page`) and validated values (`include[]` only `items` or `submission` ; `enrollment_state` only `active`).
5. Returns the exact path that will be sent to Canvas.

This file **must match** `src/data/canvas/endpoints.ts` in the extension. If a path is added in one, it is added in the other, and its purpose is documented.

5.3 The Canvas call (`canvas.ts`)

`canvasGet(env, path)` is the only function that builds a request to Canvas:

- `GET`, `Authorization: Bearer`, `Accept: application/json`, `User-Agent: FARO/0.1 (read-only)`.
- `AbortController` with the configured timeout.
- `redirect: 'manual'`: a redirect is not followed (it could carry the token to another host).
- Follows `Link: rel="next"` to the end and concatenates arrays; maximum 50 pages.
- Before each page, verifies the URL is still on `env.canvasApiUrl`.
- Errors as `CanvasError { status, path, kind: 'http' | 'timeout' | 'network' }`. No response body.

5.4 The HTTP layer (`http.ts`)

`HttpApp` on top of `node:http`:

- **Router** by method and regular expression. Unknown path → 404; known path with the wrong method → 405.
- **CORS**: if there is an `Origin` and it is not in the list → 403 `origin_not_allowed`, with no CORS headers. `OPTIONS` preflight → 204 with `GET`, `POST`, `OPTIONS`. When the server listens on loopback (`FARO_HOST = 127.0.0.1`, `localhost` or `::1`, the default), it also accepts any `chrome-extension://<32 letters a-p>` `origin` (`allowExtensionOrigins`), so the unpacked extension works without copying its id. It is turned off with `FARO_STRICT_ORIGINS=true`, and is only on when the server listens on loopback (not on e.g. `0.0.0.0`). In production: the exact id in `FARO_ALLOWED_ORIGINS` and `FARO_STRICT_ORIGINS=true`.
- **POST body**: `application/json` only (otherwise 415 `json_required`), at most 32 KB (`MAX_BODY_BYTES`; otherwise 413 `body_too_large`), valid JSON (otherwise 400 `invalid_json`). The body is never logged or echoed in an error.
- **Rate limit**: fixed window per remote address; when exceeded → 429 with `Retry-After: 60`. A sweep every minute discards old windows.
- **Headers** on every response: `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `X-FARO-Request: <id>`.
- **Log**: one JSON line per request with `id`, `method`, `path` (no query), `status`, `ms`. Never headers, bodies or query strings: a mentor message is a student's own words. Internal errors are logged with their message and the client receives 500 `internal`, never the detail.

Handlers have the shape `(req: Req) => Reply | Promise<Reply>`, with `json(status, body, headers?)` as a helper. That shape ports to Fastify unchanged.

5.5 Routes (`routes/`)

Route	Handler	Response
<code>GET /health</code>	<code>app.ts</code>	<code>{ ok, service, uptimeSeconds, canvas: { host, mode: 'live', readOnly: true } null, mentor: { provider, model } null, allowedEndpoints: [{ name, purpose }] }</code>
<code>GET /api/launch</code>	<code>launch.ts</code>	<code>{ courseId, courseName, userId, verified: false, source: 'config' 'enrollment' }</code> . Resolves <code>users/self</code> (keeps only the id) and the course from <code>FARO_COURSE_ID</code> or the first active enrolment. Cached in memory. Invalid token → 502 <code>canvas_token_rejected</code> . Without Canvas configured → 503 <code>canvas_not_configured</code> .
<code>GET /canvas/api/v1/...</code>	<code>canvas.ts</code>	Allow list → <code>canvasGet</code> → full JSON. Headers <code>X-FARO-Canvas-Pages</code> and <code>X-FARO-Canvas-Endpoint</code> . Errors: 403 (not allowed), 401/403/404 (passed through from Canvas), 502 <code>canvas_unreachable</code> , 504 <code>canvas_timeout</code> ; without Canvas configured, 503 <code>canvas_not_configured</code> .
<code>POST /api/mentor</code>	<code>mentor.ts</code>	<code>{ text, suggestions, source: 'gemini' 'guardrail', model }</code> . See 5.6.
<code>GET /lti/login</code> , <code>POST /lti/launch</code> , <code>GET /.well-known/jwks.json</code>	<code>lti.ts</code>	501 <code>lti_not_configured</code> with the missing step. The full flow specification is in the file's comments.

5.6 The mentor (`mentor.ts`, `routes/mentor.ts`)

`POST /api/mentor` answers the Mentor's open conversation with Google Gemini. Three things live in `mentor.ts` and nowhere else.

1. `sanitizeMentorRequest` — the second privacy boundary. The request is `{ message, context, history?, mode? }`. It is rebuilt from scratch:

- `context`: only the eleven `MentorContext` fields (`course`, `progress`, `next_activity`, `estimated_time`, `student_goal`, `destination`, `available_time`, `momentum`, `friction_state`, `style`, `language`). Enums are checked against their list, numbers are clamped, texts are capped (`LIMITS`); any extra field is dropped.
- `history`: the last 8 turns (`LIMITS.historyTurns`), only `role` (`student` or `faro`) and `text`.
- `mode`: `chat`, `teach`, `focus`, `recovery` or `planning` (default `chat`).
- An invalid field → `400 invalid_request`.

2. `systemPrompt(context, mode)` — the rules are the product. Rules that do not change with voice or mode: guide and never produce graded work; never label the student (no "at risk", no overdue counts); at most 90 words, plain text, at most one question; ask more than tell; calm, no cheerleading; when motivation is low, quote the destination the student wrote; only use the facts in the context; deadlines, extensions and grades belong to the institution; in a crisis, respond with care, stop the study talk and point to the university's student support or emergency services. Each mode adds its own text; for example, `TEACH MODE`: explain in at most three sentences with an everyday example, ask exactly one check question, then evaluate and adapt. The reply is requested as JSON `{ text, suggestions (≤ 3) }`; a prose reply is also accepted (`parseReply`).

3. `callGemini` — a single call. `POST {GEMINI_API_BASE}/v1beta/models/{GEMINI_MODEL}:generateContent` with the key **only in the `x-goog-api-key` header**, never in the URL; `AbortSignal.timeout(GEMINI_TIMEOUT_MS)`; the model's "thought" parts are dropped. Gemini's error body is never read into a message or a log.

Order in the route: no key → `503`; invalid request → `400`; a "do my graded work" request → a fixed server answer, `source: 'guardrail'`, **without calling the model** (no cost, same answer every time); per-client cap → `429`; then Gemini.

Codes the extension falls back to the local mentor on:

Code	When
<code>503 mentor_not_configured</code>	No <code>GEMINI_API_KEY</code>
<code>502 mentor_key_rejected</code>	Google refused the key (401/403)
<code>502 mentor_upstream</code>	Any other Gemini error
<code>502 mentor_blocked</code>	The model's safety filter stopped the reply
<code>504 mentor_timeout</code>	Gemini did not answer in time
<code>429 mentor_rate_limited</code>	<code>FARO_MENTOR_PER_MINUTE</code> exceeded for that client
<code>400 invalid_request</code>	The body failed the allow list

Plus the HTTP layer's: `413 body_too_large`, `415 json_required`.

Logging: nothing about the conversation is logged (not the message, not the reply, not the context), and the key never appears in logs or responses. The log line is the same as any route's.

5.7 Start-up (`index.ts`)

```
[faro-server] listening on http://127.0.0.1:3000
[faro-server] Accepting the unpacked FARO extension (any chrome-extension:// origin, loopback only)
[faro-server] Canvas: canvas.instructure.com (read-only, token in memory only)
[faro-server] Mentor: Gemini gemini-3.5-flash (key in memory only)
```

The second line appears only when extension origins are accepted (loopback without `FARO_STRICT_ORIGINS=true`). If Canvas or the mentor is missing, its line says `not configured`. With an invalid configuration:

```
[faro-server] Nothing to serve: set CANVAS_API_URL + CANVAS_ACCESS_TOKEN, GEMINI_API_KEY, or both
[faro-server] Copy server/.env.example to server/.env and fill it in.
```

`SIGINT` / `SIGTERM` close the server cleanly.

5.8 Tests (`test/`)

`fake-canvas.ts` is a fake Canvas that demands the Bearer, paginates with `Link` (2 rows per page to force several pages), and can answer `401` on analytics the way Canvas does for a student without the permission.

`fake-gemini.ts` is a fake Gemini: it demands the key in the header the way Google does, records every body and URL it receives so tests can check what would have left for Google, and answers whatever the test picks (JSON, prose, a safety block, an HTTP status or a delay).

`e2e.ts` starts the backend on top of those fakes and runs 38 checks: 21 for Canvas, CORS (including that an extension origin is refused when they are not enabled), logs and `.env`, and 17 for the mentor (including the call from the unpacked extension on loopback), the mentor-only mode and the configuration rules (loopback default, `FARO_STRICT_ORIGINS` and the `0.0.0.0` case). It runs with `npm test` and exits non-zero on the first failure. What it checks is, literally, what Book 1 claims.

5.9 Adding a Canvas path (procedure)

1. Justify it: which screen needs it and what minimum data it reads.
2. Add the function in `src/data/canvas/endpoints.ts` and the scope in `requiredScopes`.
3. Add the pattern and its purpose in `server/src/allowlist.ts`.
4. Declare only the fields that are read in `src/data/canvas/raw.ts`.
5. Add the fixture in `fixtures.ts` and the route in `fixtureRoutes` (`client.ts`).
6. Add the path to `fake-canvas.ts` and a check to `e2e.ts`.
7. Update Book 1 (chapter 2, data inventory) and this chapter.

If step 1 has no clear answer, the path is not added.

6. Canvas implementation, step by step: free account

This procedure connects FARO to a real Canvas in one afternoon, without asking an institution for anything. It uses **Canvas Free-for-Teacher** (`canvas.instructure.com`), which is a real Canvas instance with the same API.

You need two email addresses: one for the *teacher* account (which creates the course) and another for the *student* account (the one FARO accompanies).

6.1 Create the course (teacher account) — 15 minutes

1. Go to `https://canvas.instructure.com/register` and choose **Free for Teacher**. Complete the sign-up with the first email and confirm.
2. On the dashboard, **+ Course** (or *Start a new course*). Name it, for example *Gestión de Proyectos*, and **Create course**.
3. Create at least **two modules** (*Modules* → **+ Module**). Inside each, **+ → Assignment** → *Create Assignment*, with a name and points. Create 4 or 5 assignments in total. Optional: give one a past due date so FARO has something *overdue* to reopen.
4. **Publish** the course (the *Publish* button on the course home page) and **publish every module and assignment** (the cloud icon in green). An unpublished item does not appear for the student.
5. Note the **course id**: it is in the URL, `https://canvas.instructure.com/courses/1234567` → `1234567`.

6.2 Enrol the student — 5 minutes

Two ways:

- **Invitation:** *People* → **+ People**, type the second email, role *Student*, **Next** → **Add Users**. The student receives an email and accepts.
- **Self-enrolment:** *Settings* → *Course Details* → *More Options* → *Let students self-enroll by sharing with them a secret URL*. Save. A link appears; the student opens it.

With the student account (second email), log in to Canvas, accept the invitation, and **submit at least two assignments** (even with a one-line text). That way FARO has real progress to show.

6.3 Generate the token (student account) — 2 minutes

Logged in as the **student**:

1. **Account** (top left) → **Settings**.
2. Scroll down to **Approved Integrations** → **+ New Access Token**.
3. *Purpose*: `FARO pilot`. *Expires*: set a date (30 days is reasonable). **Generate Token**.
4. **Copy the token now**. Canvas shows it in full only at this moment.

Use the **student's** token, not the teacher's. A teacher's token has no `submission` of its own and no `module state`, and FARO would show a course with no progress. It also limits the blast radius of a leak to what that student sees about themselves (Book 1, chapter 4).

6.4 Configure and start the backend — 3 minutes

```
cd server
cp .env.example .env
```

Edit `server/.env`:

```
CANVAS_API_URL=https://canvas.instructure.com
CANVAS_ACCESS_TOKEN=<paste the student's token here>
FARO_COURSE_ID=1234567
```

Start it:

```
npm start          # or, from the project root: npm run server
```

You should see:

```
[faro-server] listening on http://127.0.0.1:3000
[faro-server] Accepting the unpacked FARO extension (any chrome-extension:// origin, loopback only)
[faro-server] Canvas: canvas.instructure.com (read-only, token in memory only)
[faro-server] Mentor: not configured (the extension uses its local mentor)
```

Check from another terminal:

```
curl http://127.0.0.1:3000/health
curl http://127.0.0.1:3000/api/launch
```

`/api/launch` must return your `courseId`, the course name and the student's numeric `userId`. If it returns `canvas_token_rejected`, the token was mis-copied or has expired.

On Windows use PowerShell: `Copy-Item .env.example .env` instead of `cp`, and `Invoke-WebRequest http://127.0.0.1:3000/health | Select-Object -Expand Content` instead of `curl` (or install curl).

6.5 Build the extension in live mode — 3 minutes

In the project **root** create `.env.local`:

```
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

Option A, dev server (fastest to iterate):

```
npm run dev
```

Open `http://localhost:5173`. That origin is already in `FARO_ALLOWED_ORIGINS` by default.

Option B, extension loaded in Chrome:

```
npm run build
```

Chrome → `chrome://extensions` → *Developer mode* → *Load unpacked* → the `dist/` folder.

In local development there is no need to copy the extension id: while the backend listens on loopback (`FARO_HOST=127.0.0.1`, the default), it accepts any `chrome-extension://` origin, and says so at start-up with *"Accepting the unpacked FARO extension (any chrome-extension:// origin, loopback only)"*. **Optional** (and mandatory off the development machine): note the **extension id** Chrome assigns, add it to `server/.env` and turn on strict mode:

```
FARO_ALLOWED_ORIGINS=http://localhost:5173,chrome-extension://<id>
FARO_STRICT_ORIGINS=true
```

Restart the backend.

Two terminals from the root: `npm run server` starts the backend (= `npm --prefix server run dev`, with auto-reload) and, in the other, `npm run dev` (browser) or `npm run build` and load `dist/` unpacked (extension).

6.6 Verify — 2 minutes

1. Open FARO. Complete the purpose (three questions).
2. **Home** must show your real course name and a progress that matches the student's submissions.
3. **Profile** → **Canvas**: the badge says **Live**, *Course 1234567 · Gestión de Proyectos* appears, and the note says which backend the calls go through. Open the log: four calls with status `200`.
4. In the backend terminal, four log lines with `status: 200` and **no query strings or token**.

If the `/analytics/.../activity` path shows `401` in the log and the panel shows the note about analytics: that is the expected behaviour for a student account in Free-for-Teacher. FARO uses your submission dates as activity. It is not an error.

6.7 Break it on purpose (optional, recommended)

- Revoke the token in Canvas (*Approved Integrations* → *delete*). Restart the backend: `canvas_token_rejected`. FARO shows the error with *Retry*, not an empty course.
- Ask for a forbidden path: `curl http://127.0.0.1:3000/canvas/api/v1/courses/1234567/enrollments` → `403 path_not_allowed`, and **no new line** in the Canvas log: it never reached the network.
- Change `FARO_ALLOWED_ORIGINS` to exclude `localhost:5173` and reload the dev server: `403 origin_not_allowed`.

6.8 Common problems

Symptom	Likely cause	Fix
<code>CANVAS_API_URL</code> must use <code>https://</code>	You used <code>http://</code>	Fix the URL
<code>canvas_token_rejected</code>	Token mis-copied, expired or revoked	Generate a new one
<code>no_active_course</code>	The student did not accept the invitation or the course is unpublished	Accept the invitation / publish the course
Home shows 0% with assignments present	The assignments are in no module, or the token is the teacher's	Add the assignments to modules; use the student token
FARO does not load and the console says <code>CORS</code>	The origin is not in <code>FARO_ALLOWED_ORIGINS</code> and extension origins are not accepted: <code>FARO_STRICT_ORIGINS=true</code> , or the backend listens off loopback (e.g. <code>0.0.0.0</code>)	Add <code>chrome-extension://<id></code> to <code>FARO_ALLOWED_ORIGINS</code> and restart
<code>ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX</code>	Node older than 22.18	Update Node
Modules appear with no state (all <code>unlocked</code>)	Teacher token	Use the student token
<code>Nothing to serve: ...</code> at start-up	<code>server/.env</code> has neither Canvas nor <code>GEMINI_API_KEY</code>	Configure at least one
Every Mentor reply carries the note "Gemini could not answer..."	Key rejected (<code>502 mentor_key_rejected</code>), a model name that does not exist (<code>502 mentor_upstream</code>) or, with <code>VITE_MENTOR_MODE=gemini</code> , backend down or without a key (<code>503</code>)	Check <code>/health</code> (<code>mentor</code> must not be <code>null</code>), the key and <code>GEMINI_MODEL</code>
The Mentor footer says "Local mentor" although you configured Gemini	The server is not running, has no <code>GEMINI_API_KEY</code> , or the extension was built with <code>VITE_MENTOR_MODE=local</code>	Start the server with the key (<code>/health</code> must show <code>mentor</code>), remove <code>VITE_MENTOR_MODE=local</code> and reopen the Mentor

6.9 Turning on the Gemini mentor (optional) — 3 minutes

The mentor works without this (local mentor). For Gemini to answer:

1. Create a key in Google AI Studio. **With real students, only from a Google Cloud project with active billing**; the free tier is only for testing with made-up data (Book 1, chapter 7.8).
2. Add it to `server/.env` (never to the root `.env.local`):

```
GEMINI_API_KEY=<your key>
GEMINI_MODEL=gemini-3.5-flash
```

Model names change: check the current one at `ai.google.dev/gemini-api/docs/models`.

3. Start the backend (`npm run server` from the root, or `npm run dev` / `npm start` inside `server/`). The mentor line must say `Mentor: Gemini <model> (key in memory only)`. Canvas is optional: with only the key, the backend starts mentor-only and the Canvas routes answer `503 canvas_not_configured`.
4. Open FARO (`npm run dev` or the loaded extension). No rebuild and no `.env.local` are needed: in the default mode (`VITE_MENTOR_MODE=auto`), every time the Mentor opens the extension asks the backend at `VITE_FARO_API_URL` (by default `http://127.0.0.1:3000`) for `GET /health` and, if the server reports a mentor, uses Gemini. `VITE_MENTOR_MODE=gemini` always tries it; `local` prevents it.

5. Check: the Mentor footer says *"Answers by Gemini, through FARO's server..."*, and when you type an open message the reply arrives with the violet ring on the avatar and **without** the note *"Gemini could not answer..."*.
6. Stop the backend and reopen the Mentor: the footer must say *"Local mentor..."* and the local mentor must answer.

This is the step the team must do with its own key: in development, the path was only verified against a fake Gemini (chapter 8.2b).

7. Canvas implementation, step by step: institution with LTI 1.3

This is the real deployment path. A browser extension cannot be installed institutionally; an LTI tool can. That is why the plan is **LTI 1.3 first, the extension as an optional layer** for those who prefer it.

What follows has two parts: what the institution does (register the tool) and what the FARO team does (implement three routes that answer 501 today). Neither is done yet; both are specified.

7.1 What FARO asks of the institution

A Canvas administrator creates a **Developer Key** → **LTI Key** with these values:

Field	Value
Method	Manual Entry
Title	FARO
Redirect URIs	<code>https://<faro>/lti/launch</code>
Target Link URI	<code>https://<faro>/lti/launch</code>
OpenID Connect Initiation Url	<code>https://<faro>/lti/login</code>
JWK Method	Public JWK URL: <code>https://<faro>/.well-known/jwks.json</code>
LTI Advantage Services	☒ <code>contextmembership.readonly</code> (NRPS) · ☒ <code>result.readonly</code> (AGS) · ☐ <code>score</code> · ☐ <code>lineitem</code>
Placements	Course Navigation
Privacy Level	Anonymous or Public as the institution decides (see 7.4)

And a **Developer Key** → **API Key** with exactly the scopes from `src/data/canvas/endpoints.ts` (`requiredScopes`), with *Enforce Scopes* enabled:

```
url:GET|/api/v1/courses/:id
url:GET|/api/v1/courses/:course_id/modules
url:GET|/api/v1/courses/:course_id/assignments
url:GET|/api/v1/courses/:course_id/analytics/users/:student_id/activity
url:GET|/api/v1/courses
url:GET|/api/v1/users/self
```

The administrator hands FARO: `client_id`, `deployment_id`, the instance URL and Canvas's JWKS URL (`https://<institution>.instructure.com/api/lti/security/jwks`).

7.2 What FARO implements (the three routes)

All with `node:crypto`, no dependencies. The specification is in `server/src/routes/lti.ts`.

GET /lti/login — third-party OIDC initiation. Receives `iss`, `login_hint`, `target_link_uri`, `client_id`, `lti_message_hint`. Generates a random `state` and `nonce`, stores them with a short expiry (memory or database), and redirects (302) to `https://<institution>.instructure.com/api/lti/authorize_redirect` with `scope=openid`, `response_type=id_token`, `response_mode=form_post`, `prompt=none`, `redirect_uri`, `client_id`, `login_hint`, `state`, `nonce`, `lti_message_hint`.

POST /lti/launch — `id_token` verification.

1. Compares `state` with the stored one; consumes it.
2. Decodes the JWT; takes `kid` from the header; obtains the public key from Canvas's JWKS (cached); verifies the RS256 signature with `crypto.verify`.
3. Verifies `iss` (the instance), `aud` (the `client_id`), `exp`, `iat`, and that `nonce` is the issued, unused one.
4. Reads `https://purl.imsglobal.org/spec/lti/claim/context` → `id` (course) and `sub` (person). Optionally the NRPS and AGS endpoints from their respective claims.
5. Creates a FARO session (`HttpOnly`; `Secure`; `SameSite=None` cookie) bound to (course, person, instance).
6. Redirects to the FARO interface.

GET /.well-known/jwks.json — FARO's public key. FARO generates an RSA key pair on deployment (stored as a secret), publishes the public one here, and signs with the private one the `client_assertion` (RFC 7523) it sends to

`https://<institution>.instructure.com/login/oauth2/token` with `grant_type=client_credentials` to obtain LTI service tokens (NRPS/AGS).

7.3 Per-student API token (OAuth2)

For the six REST paths, with the API Key:

1. After the launch, if there is no token for that person, FARO redirects to `https://<institution>.instructure.com/login/oauth2/auth?client_id=...&response_type=code&redirect_uri=https://<faro>/oauth/callback&scope=<the six scopes>&state=...`.
2. Canvas asks the student for consent (once) and returns a `code`.
3. FARO exchanges it at `POST /login/oauth2/token` for an `access_token` (1 hour) and a `refresh_token`.
4. Both are stored **only in the backend**, encrypted at rest, bound to the person.
5. On expiry, FARO renews with the `refresh_token`. On consent withdrawal, `DELETE /login/oauth2/token`.

With this, `CANVAS_ACCESS_TOKEN` disappears from `.env`: the backend goes from one shared token to one token per person.

7.4 What changes in the backend

Today (pilot)	Production
<code>GET /api/launch</code> infers course and person from the <code>.env</code> token	Reads them from the session created by <code>/lti/launch</code> ; <code>verified: true</code>
One token for everything	One OAuth2 token per person, stored encrypted
Authorisation by origin (CORS) and localhost	Authorisation by session (cookie) + CORS
FARO state in the browser	PostgreSQL: purpose, sessions, preferences, tokens
Local mentor, or Gemini via <code>POST /api/mentor</code> with a key in <code>.env</code>	The same <code>POST /api/mentor</code> , with a key from an institutional project with active billing, terms reviewed by legal, and the LTI session as authorisation

`allowlist.ts`, `canvas.ts` and the `/canvas/*` routes **do not change**: the allow list and pagination are the same with any token.

Privacy Level of the LTI Key. *Anonymous* sends no name or email in the launch; FARO does not need them, so it is the option consistent with Book 1. *Public* would send them; FARO would ignore them, but better not to receive them.

7.5 Suggested order of work

1. Deploy the backend behind TLS (own domain, valid certificate). Without this, Canvas does not accept the tool.
2. Generate FARO's key pair; publish `/.well-known/jwks.json`.
3. Implement `/lti/login` and `/lti/launch` against a test course in the institution's instance (or in Free-for-Teacher, which also admits LTI Keys from the teacher account).
4. Implement the OAuth2 flow from 7.3.
5. Replace the launch with the session in `/api/launch`.
6. Add PostgreSQL and move FARO's state.
7. Register the tool at account level; pilot with one course.

7.6 The extension in this scenario

It keeps existing as an option: whoever prefers the side panel over the Canvas iframe installs it, and the extension calls the same backend with the same session (the cookie travels if `FARO_ALLOWED_ORIGINS` includes the exact extension id, with `FARO_STRICT_ORIGINS=true`, and the cookie is `SameSite=None`). The extension code does not change; only where `VITE_FARO_API_URL` points changes.

8. Verification and tests

8.1 What is verified automatically

`cd server && npm test` starts a fake Canvas with real pagination, a fake Gemini (`server/test/fake-gemini.ts`) and the backend on top, and runs these 38 checks:

```
ok health names the Canvas host and never the token
ok launch resolves the token owner and the first active course
ok course proxy returns the Canvas object
ok modules are paginated by Canvas and merged by the backend
ok assignments carry submissions across three pages
ok activity is forwarded for the token owner
ok a path outside the allow list is refused locally (no Canvas call)
ok a write is refused even on an allowed path
ok an unknown query parameter is refused
ok path traversal is refused
ok a browser origin outside the list gets 403 and no data
ok an extension origin is refused unless extension origins are enabled
ok CORS preflight succeeds for an allowed origin
ok responses carry no-store and nosniff
ok Canvas 404 is passed through as 404
ok the token was sent to Canvas on every call, and only there
ok log lines carry no query strings
ok rate limit trips after the configured burst
ok Canvas 401 on analytics is passed through so the client can fall back
ok a wrong token is reported as rejected, not as a generic failure
ok .env parser handles comments, quotes and CRLF
ok mentor: the unpacked extension can call it when extension origins are on (loopback dev)
ok mentor: answers with Gemini and returns text + suggestions
ok mentor: the key goes in a header, never in the URL
ok mentor: extra fields in the context never reach Gemini
ok mentor: teach mode reaches the system prompt; thought parts are dropped
ok mentor: history is trimmed to the last 8 turns
ok mentor: "do my homework" is refused before any model call
ok mentor: a bad context is rejected with 400
ok mentor: a non-JSON body gets 415 and a huge body 413
ok mentor: prose instead of JSON still becomes a reply
ok mentor: a rejected key and a safety block map to fallback codes
ok mentor: a slow model times out with 504
ok mentor: per-client cap answers 429 before calling the model
ok mentor: the key and the conversation never appear in the log
ok mentor: without a key the route answers 503 so the extension falls back
ok mentor-only server: Canvas routes say canvas_not_configured
ok config: half a Canvas setup is refused, mentor-only is accepted

38 checks passed
```

One of the first checks that a `chrome-extension://` origin is refused when the server does not accept extension origins. The last 17 cover the mentor: that the unpacked extension can call it when the server accepts extension origins (loopback development); that Gemini's reply becomes `{ text, suggestions }`; that the key goes only in the header; that extra context fields never reach Gemini; that Teach me mode reaches the prompt; that history is trimmed to 8 turns; that "do my homework" is answered without calling the model; the `400`, `413` and `415`; that a prose reply is accepted; that a rejected key gives `mentor_key_rejected` without echoing Google's body; the safety block; the `504`; the per-client `429`; that neither the key nor the conversation appears in the log; the `503` without a key; a mentor-only server's `canvas_not_configured` and `/health`; and the configuration rules (including the loopback default, `FARO_STRICT_ORIGINS` and the `0.0.0.0` case).

It uses no test framework: `node:assert` and a ten-line `check(name, fn)`. The process exits non-zero on the first failure, so it works as a CI step.

8.2 What was verified of the extension's mapping

In addition, the `src/data/canvas/` layer was compiled and run in Node against the same fake Canvas and the real backend, in the four scenarios that matter:

Scenario	Expected and obtained result
Mock	16 activities, 7 completed, 4 fixture calls, launch <code>fixture</code> , study profile present
Http through the backend	Course GP-101, 3 modules (the third <code>locked</code>), 5 assignments (orphan 999 excluded), 3 with a submission, correct <code>module_id</code> , <code>course_id</code> from the live course, 20 min default, 3 activity events ordered newest first, no study profile, <code>activityFallback = false</code> , 4 calls with 200

Scenario	Expected and obtained result
Analytics forbidden (401)	<code>activityFallback = true</code> , activity derived from 3 submissions, the analytics path with 401 in the log, no call to <code>/enrollments</code>
Backend down	<code>getCourseSnapshot()</code> rejects; it never returns an empty course

This script is not in the repository because it depends on compiling the extension without Vite; the scenarios are described here so they can be reproduced.

8.2b The Gemini mentor, end to end

The full path (extension with `VITE_MENTOR_MODE=gemini` → `POST /api/mentor` → Gemini) was verified in the browser **against the fake Gemini**, including the fallback: with the fake down, the extension answered with the local mentor and the note *"Gemini could not answer just now, so the local mentor did"*.

With the default mode (`VITE_MENTOR_MODE=auto`) it was also verified in the browser preview: with the backend and the fake Gemini running, the footer said *"Answers by Gemini..."* and a Gemini reply arrived; with the backend stopped, the footer said *"Local mentor..."* and the local mentor answered.

It has not been exercised against Google's real API from the development environment: the network policy blocked it. The team validates it with its own key (chapter 6.9 and the README). Model names change: check `GEMINI_MODEL` at `ai.google.dev/gemini-api/docs/models` before trying.

8.3 Manual verification in the interface

With the fixtures (mock mode), the app must show exactly:

Data	Value
Total activities	16
Completed	7
Progress	44%
Remaining	9 activities, 4 h 35 min
Days until the close	12
Days without activity	5
Friction state	FRICTION
Module 3	started
Module 5	locked

And in **Profile** → **Canvas**: *Simulated* badge, four GET paths, two refused scopes, a log with four `200` entries (~140 ms).

8.4 Break it on purpose

Three experiments with the fixtures, each with a result that must change:

1. **DAYS_INACTIVE from 5 to 9** → `DISCONNECTION` state, Home changes tone, Recovery offers a comeback mission. None of this is written in the views: it comes from the `disconnectionDays: 7` threshold.
2. **DAYS_UNTIL_COURSE_ENDS from 12 to 3** → *not realistic* verdict with all three strategies marked infeasible and work in overflow.
3. **A typo in a path in `endpoints.ts`** → 404 in the panel's log. The mock does not resolve by coincidence.

8.5 Typecheck

- Root: `npm run typecheck`. Requires a prior `npm install` (React, Vite). If it prints nothing after the two header lines, it passed.
- `server/: npm install && npm run typecheck`. Installs only `typescript` and `@types/node` as dev dependencies; the running backend still has no dependencies.

`en.ts` typed against `es.ts` makes a missing key in one language a compile error, not a blank text in the demo.

8.6 What is not tested yet

- The React interface against a real backend in a browser (the tests above cover the data layer, not the render). Covered by the procedure in chapter 6.
- The LTI routes (they answer 501 by design).
- The mentor against Gemini's real API (see 8.2b).
- The new `lib/` logic (`timeSession` , `rewardPath` , `teach` , `rhythm`) has no automated tests of its own.
- Behaviour with large courses (more than 100 modules or assignments, i.e. more than one page of real Canvas). Pagination is tested with the fake Canvas; it is worth repeating against Free-for-Teacher with `per_page=2` forced in a test.

9. What is real and what is simulated

This list exists so that nobody — not the jury, not an institution, not the team six months from now — believes the prototype does something it does not.

Real and working

Component	State
Backend with token, allow list, pagination, CORS, rate limit, logs without sensitive data	Implemented; 38 automated checks in total
Gemini mentor: <code>POST /api/mentor</code> , re-checked context, 8-turn history, guardrail answer without the model, fallback codes, key and conversation kept out of logs, mentor-only mode	Implemented; verified against a fake Gemini (17 of the 38 checks) and end to end in the browser against that fake, including fallback to the local mentor and auto-detection (<code>VITE_MENTOR_MODE=auto</code> : with the backend, Gemini answers; with the backend stopped, the local mentor). Not exercised against Google's real API from the development environment (the network policy blocked it); the team validates it with its own key. Model names change: check <code>GEMINI_MODEL</code>
Local mentor and structured moments (check-in, Focus, Way back, Teach me with the demo bank, weekend plan, points towards TecmiRewards)	Implemented; computed in the extension from real data, never by a model
How much time do you have?, the <i>Your plan</i> card, the rhythm week in Progress	Implemented (<code>timeSession.ts</code> , <code>rhythm.ts</code>)
Extension in <code>http</code> mode against the backend, with no credentials and no permission for <code>instructure.com</code>	Implemented; data layer verified end-to-end
Canvas-to-FARO mapping (modules from items, done from submission, orphans excluded)	Implemented and verified with paginated responses
Activity fallback from submissions when Canvas denies analytics, with an on-screen note	Implemented and verified
Error state with retry when the backend does not answer	Implemented
Friction engine, journey, next best action, planner, points, achievements, feed ranking, SOS routing, mentor contract	Implemented as pure functions; configurable thresholds
<code>es/en i18n</code> verified by the compiler	Implemented
Review cards	Flow and rules implemented; the question bank is hand-written for the demo course
Impact (formerly Rewards)	Rules implemented (semester track, tiers, one redemption per semester, path to the next milestone); amounts, previous courses (16 activities and 6 modules each) and redemption simulated. Prototype integration with TecmiRewards: FARO counts points and issues nothing; we found no public TecmiRewards API, so the integration must be validated with Tecmilenio
Live request log in Profile	Implemented

Ready to run, pending credentials

Component	What is missing
Connection to Canvas Free-for-Teacher	A student token and the course id (chapter 6, 30 minutes)
Mentor with Gemini's real API	A key from a project with active billing in <code>server/.env</code> ; in the default mode (<code>auto</code>) the extension detects it by itself (chapter 6.9)

Simulated, and labelled as such

Component	What is missing
Course data in mock mode	Comes from <code>fixtures.ts</code> . The badge says <i>Simulated</i> .
Launch	<code>verified: false</code> in both modes. Live, course and person are inferred from the token; the panel says so.
Institutional account (SSO)	Does not exist. The screen labels it as simulated.
Community	Classmates, posts, rooms and mission are sample data. Rooms run on an accelerated clock for the demo.
Study profile (measured rhythm)	Mock only (35 min/session, 4 days/week). Live, absent and declared.
<code>estimated_minutes</code>	Does not exist in Canvas. Own table in mock; 20 min default live.
Teach me lessons without Gemini	They come from the question bank hand-written for the demo course; other courses have no local lessons.

Designed, with the specification in the code, not implemented

Component	Where the specification is
LTI 1.3: <code>/lti/login</code> , <code>/lti/launch</code> , <code>/well-known/jwks.json</code>	<code>server/src/routes/lti.ts</code> ; chapter 7
OAuth2 per student	Chapter 7.3; scopes in <code>endpoints.ts</code>
PostgreSQL and cross-device sync	Project instructions; chapter 7.4
Warning on the destination sentence	Book 1, chapter 5.2

The honesty rule

No validation figure in this project is invented. Where there is no measurement, the documents say *to be measured*. The friction thresholds are prototype rules, configurable, and are not presented as scientifically validated. What is claimed as implemented has code and, when verifiable, a test that can be run.